

CHAPTER -4

FUNCTIONAL DEPENDENCIES AND ANALYZE REDUNDANCY AND DATA ANOMALIES

INTRODUCTION

Functional Dependencies

A Functional Dependency (FD) describes the relationship between attributes in a database. It states that the value of one attribute (or a set of attributes) uniquely determines the value of another attribute. Functional dependencies help identify the correct primary keys and are essential for normalizing a database to reduce redundancy and maintain data consistency.

Why Functional Dependencies are Important

- Identify Primary Keys
- Reduce Data Redundancy
- Avoid Data Anomalies
- Improve Data Integrity
- Support Database Normalization
- Ensure Efficient Database Design

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address

Redundancy Analysis

Customer information is stored only once in the Customer table. Orders store only the Customer_ID, preventing duplicate customer records.

Data Anomalies

- Insert Anomaly: A new customer can be added without creating an order.
- Update Anomaly: Updating a customer's phone number requires changes only in the Customer table.
- Delete Anomaly: Deleting an order will not remove customer information.

Conclusion

The Customer table is well-structured, eliminates redundant customer data, and ensures consistent customer information throughout the database.

2. SELLER TABLE

Functional Dependency

$\text{Seller_ID} \rightarrow \text{Seller_Name, Store_Name, Email, Phone_Number, Address}$

Redundancy Analysis

Seller details are stored only once in the Seller table. Products reference Seller_ID, avoiding duplicate seller information.

Data Anomalies

- Insert Anomaly: A new seller can be registered without adding products.
- Update Anomaly: Updating seller details requires changes only in the Seller table.
- Delete Anomaly: Deleting a product will not remove seller information.

Conclusion

The Seller table maintains unique seller records and supports efficient management of sellers without data duplication.

3. CATEGORY TABLE

Functional Dependency

$\text{Category_ID} \rightarrow \text{Category_Name, Category_Description}$

Redundancy Analysis

Category information is stored once and linked to products using Category_ID.

Data Anomalies

- Insert Anomaly: A new category can be created without products.
- Update Anomaly: Updating a category name affects only the Category table.
- Delete Anomaly: Deleting a product does not remove category information.

Conclusion

The Category table provides organized product classification and prevents repeated category information.

4. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Description, Price, Stock, Image, Seller_ID, Category_ID

Redundancy Analysis

Product details are stored only once. Seller and category information are referenced through foreign keys.

Data Anomalies

- Insert Anomaly: A product can be added only after assigning a valid seller and category.
- Update Anomaly: Updating product details requires changes only in the Product table.
- Delete Anomaly: Deleting a product will not remove seller or category information.

Conclusion

The Product table efficiently manages jewellery products while maintaining relationships with sellers and categories.

5. CART TABLE

Functional Dependency

$\text{Cart_ID} \rightarrow \text{Customer_ID}, \text{Product_ID}, \text{Quantity}$

Redundancy Analysis

The Cart table stores only product references and quantity instead of duplicate customer or product information.

Data Anomalies

- Insert Anomaly: Cart items can be added only for existing customers and products.
- Update Anomaly: Quantity updates affect only the Cart table.
- Delete Anomaly: Removing cart items does not affect customer or product records.

Conclusion

The Cart table supports shopping activities while minimizing data redundancy through foreign keys.

6. ORDER TABLE

Functional Dependency

$\text{Order_ID} \rightarrow \text{Customer_ID}, \text{Order_Date}, \text{Total_Amount}, \text{Order_Status}$

Redundancy Analysis

The Order table stores only order details while referencing customer information using Customer_ID.

Data Anomalies

- Insert Anomaly: Orders can be created only for registered customers.
- Update Anomaly: Updating the order status affects only the Order table.
- Delete Anomaly: Deleting an order does not remove customer information.

Conclusion

The Order table accurately records purchase transactions and maintains data consistency.

7. ORDER_ITEM TABLE

Functional Dependency

Order_Item_ID $\rightarrow$ Order_ID, Product_ID, Quantity, Price

Redundancy Analysis

The Order_Item table stores products associated with each order without duplicating product details.

Data Anomalies

- **Insert Anomaly:** Order items require valid orders and products.
- **Update Anomaly:** Quantity or price updates affect only the Order_Item table.
- **Delete Anomaly:** Deleting an order item does not remove product or order information.

Conclusion

The Order_Item table efficiently handles multiple products within a single order and maintains accurate order details.

8. PAYMENT TABLE

Functional Dependency

Payment_ID $\rightarrow$ Order_ID, Payment_Method, Payment_Status, Payment_Date

Redundancy Analysis

Payment records are stored separately and linked with orders using Order_ID.

Data Anomalies

- **Insert Anomaly:** Payments can be recorded only after an order exists.
- **Update Anomaly:** Payment status updates affect only the Payment table.

- **Delete Anomaly:** Deleting payment information does not remove order records.

Conclusion

The Payment table ensures secure and organized management of payment transactions.

9. SHIPMENT TABLE

Functional Dependency

Shipment_ID → Order_ID, Delivery_Address, Shipment_Status, Delivery_Date

Redundancy Analysis

Shipment information is maintained separately while referencing orders through Order_ID.

Data Anomalies

- **Insert Anomaly:** Shipment records require an existing order.
- **Update Anomaly:** Shipment status changes affect only the Shipment table.
- **Delete Anomaly:** Deleting shipment details does not remove order information.

Conclusion

The Shipment table enables efficient delivery tracking while maintaining data integrity.

10. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text, Review_Date

Redundancy Analysis

Reviews are stored independently and linked to customers and products using foreign keys.

Data Anomalies

- **Insert Anomaly:** Reviews can be added only for existing customers and products.
- **Update Anomaly:** Editing a review affects only the Review table.

- Delete Anomaly: Deleting a review does not remove customer or product data.

Conclusion

The Review table supports customer feedback while ensuring product and customer information remains consistent.

11. INVENTORY TABLE

Functional Dependency

Inventory_ID → Seller_ID, Product_ID, Stock_Quantity, Last_Updated

Redundancy Analysis

Inventory records are maintained separately without repeating seller or product details.

Data Anomalies

- Insert Anomaly: Inventory records require existing sellers and products.
- Update Anomaly: Updating stock quantity affects only the Inventory table.
- Delete Anomaly: Deleting inventory records does not remove seller or product information.

Conclusion

The Inventory table helps maintain accurate stock levels and supports efficient inventory management.

12. ADMINISTRATOR TABLE

Functional Dependency

Admin_ID → Admin_Name, Email, Phone_Number

Redundancy Analysis

Administrator information is stored only once, avoiding duplicate administrative records.

Data Anomalies

- Insert Anomaly: A new administrator can be added independently.
- Update Anomaly: Administrator details are updated only in the Administrator table.
- Delete Anomaly: Deleting an administrator does not affect business data.

Conclusion

The Administrator table centralizes administrative information and supports effective management of the marketplace.